

# Selective Fault-Tolerant RV32E RISC-V Processor with Error-Correcting Register File and Adaptive Redundancy Control

## Project Design & Architecture Document (Version 2.0)

---

### 1. Introduction

Modern embedded systems such as drones, satellites, autonomous robots, and industrial controllers often operate in environments where hardware reliability is critical. In these systems, even a single incorrect computation caused by radiation, electromagnetic interference (EMI), or electrical noise can lead to system failure.

Traditional processors are designed primarily for performance and area efficiency, with limited protection against transient hardware faults. Highly reliable processors used in aerospace and defense applications typically employ expensive redundancy techniques, making them unsuitable for resource-constrained embedded systems.

This project proposes a **Selective Fault-Tolerant RV32E RISC-V Processor**, which improves reliability while minimizing unnecessary hardware overhead. Instead of protecting the entire processor, only the components most vulnerable to computational errors are hardened using targeted fault-tolerance mechanisms.

The processor combines:

- A custom-built **5-stage pipelined RV32E processor**
- **SEC-DED (Single Error Correction – Double Error Detection)** protection for the register file
- **Triple Modular Redundancy (TMR)** on the Execute stage
- A **self-checking majority voting mechanism**
- **Adaptive redundancy control** for power-efficient operation

The result is a processor capable of balancing **performance, reliability, area, and power consumption**, making it suitable for safety-critical embedded applications.

---

### 2. Project Objective

The primary objective of this project is to design and verify a **fault-tolerant RISC-V processor** capable of maintaining correct operation even when transient hardware faults occur.

Unlike conventional approaches that replicate the entire processor, this design selectively protects only the components where faults are most likely to affect computation.

The processor therefore aims to:

- Execute the RV32E instruction set correctly.
- Detect and correct memory errors occurring inside the register file.
- Mask computational faults occurring during instruction execution.
- Reduce power consumption by enabling redundancy only when required.
- Demonstrate the complete ASIC implementation flow using Cadence tools.

---

### 3. Why RV32E?

This project is based on the **RV32E** variant of the RISC-V ISA.

RV32E is a reduced version of RV32I designed specifically for embedded applications.

Instead of 32 general-purpose registers, RV32E contains only **16 registers**, reducing:

- silicon area
- power consumption
- register file complexity

These characteristics make RV32E particularly suitable for battery-powered systems such as drones and IoT devices.

The processor still supports the standard RISC-V programming model while requiring fewer hardware resources.

---

### 4. Overall Processor Architecture

The processor follows the classic **5-stage pipelined architecture**.

```
graph TD; A[Instruction Fetch] --> B[Instruction Decode]; B --> C[Execute]; C --> D[Memory]; D --> E[Write Back];
```

Instruction Fetch

|

Instruction Decode

|

Execute

|

Memory

|

Write Back



Each instruction passes through these five stages, allowing multiple instructions to be processed simultaneously.

This improves throughput significantly compared to a single-cycle processor.

---

## 5. Five Pipeline Stages

### Stage 1 — Instruction Fetch (IF)

The Fetch stage retrieves instructions from Instruction Memory.

Main components:

- Program Counter (PC)
- PC + 4 Adder
- Instruction Memory

Responsibilities:

- Read the next instruction.
  - Update the Program Counter.
  - Pass the instruction to the Decode stage.
- 

### Stage 2 — Instruction Decode (ID)

The Decode stage determines what the instruction is supposed to do.

Main components:

- Register File
- Immediate Generator
- Control Unit

Responsibilities:

- Decode the instruction opcode.
  - Read operands from registers.
  - Generate immediate values.
  - Produce control signals for later stages.
- 

### Stage 3 — Execute (EX)

The Execute stage performs all arithmetic and logical operations.

Examples include:

- Addition
- Subtraction
- Logical operations
- Address calculation
- Branch target calculation

Unlike conventional processors, this stage contains the project's primary fault-tolerance mechanism.

---

## Stage 4 — Memory (MEM)

Handles communication with Data Memory.

Responsibilities include:

- Load operations
  - Store operations
  - Branch decision handling
- 

## Stage 5 — Write Back (WB)

Writes computation results back into the Register File.

Possible write-back sources:

- ALU output
  - Memory output
  - PC + 4
- 

# 6. Selective Fault Tolerance

Traditional fault-tolerant processors often duplicate or triplicate the **entire processor**.

Although effective, this approach significantly increases:

- silicon area
- power consumption
- design complexity

Instead, this project adopts **Selective Fault Tolerance**.

Only the hardware blocks that directly affect computation are protected.

The processor therefore focuses protection on:

- Register File
- Execute Stage

This significantly reduces hardware overhead while maintaining strong protection against transient faults.

---

## 7. SEC-DED Protected Register File

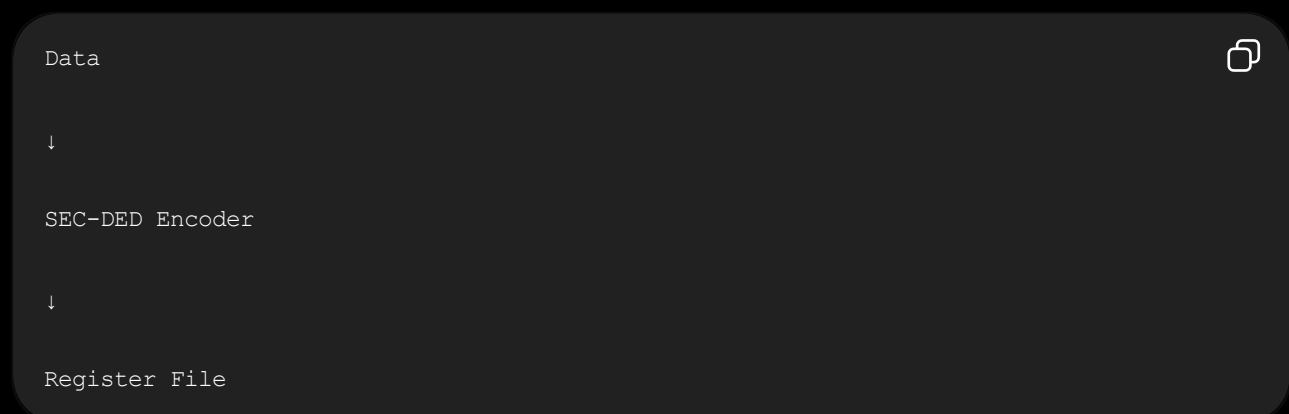
The Register File stores all temporary data used during program execution.

Since every instruction depends on register values, corruption of a register can propagate incorrect results throughout the processor.

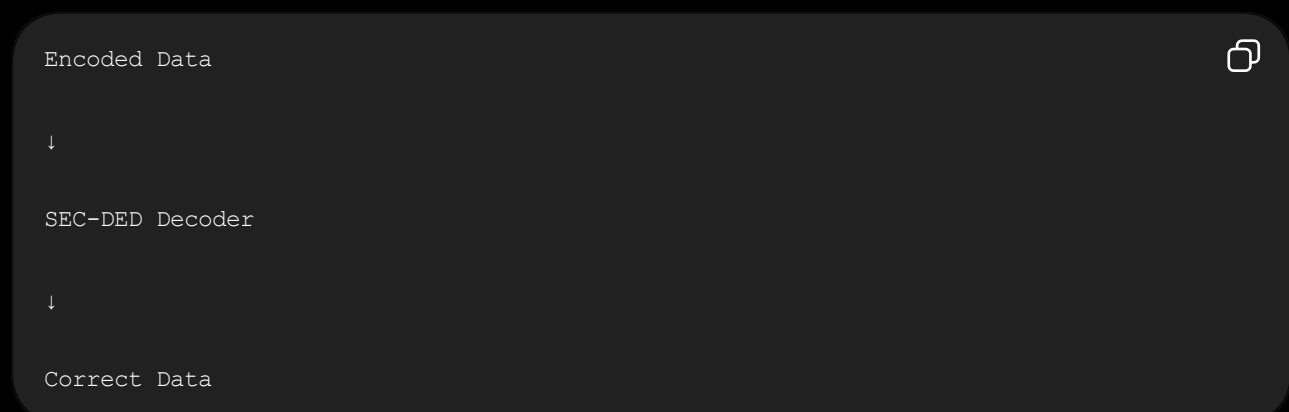
To improve reliability, each stored register value is protected using **SEC-DED (Single Error Correction – Double Error Detection)**.

### How it works

When data is written:



When data is read:



## Capabilities

Single-bit error

→ Automatically corrected.

Double-bit error

→ Detected and reported.

This mechanism prevents transient register corruption from affecting processor execution.

---

## 8. Triple Modular Redundancy (TMR)

The Execute stage performs arithmetic operations.

Because incorrect ALU outputs directly affect processor correctness, this stage is protected using **Triple Modular Redundancy (TMR)**.

Instead of one Execute unit:

Execute



the processor contains:

EX0



EX1

EX2

All three Execute units receive identical inputs and perform exactly the same computation simultaneously.

---

## 9. Majority Voting (AMVU)

After computation, the outputs of EX0, EX1, and EX2 are compared.

Example:

EX0 → 250



EX1 → 250

The majority output is:

250



The incorrect result is ignored.

This allows the processor to continue operating correctly even if one Execute unit experiences a transient hardware fault.

---

## 10. Self-Checking Majority Voter

A traditional TMR system assumes the voter itself never fails.

However, if the voter becomes faulty, the entire protection mechanism becomes unreliable.

To eliminate this weakness, the project introduces a **self-checking majority voter**.

The voting logic is duplicated:

EX0

EX1

EX2

↓

Majority Voter A

↓

Majority Voter B

↓

Comparator



Both voters independently compute the majority result.

If their outputs differ, a fault is detected within the voting circuitry itself.

This improves overall system reliability by ensuring that the fault-tolerance mechanism is also monitored.

---

## 11. Adaptive Redundancy Control

Running three Execute stages continuously increases power consumption.

To improve efficiency, the processor supports two operating modes.

## Simplex Mode

Only EX0 Active



Characteristics:

- Lowest power consumption
- Minimum hardware activity
- Suitable for normal operation

The unused Execute units are disabled using:

- Clock gating
- Operand isolation

---

## TMR Mode

EX0



EX1

EX2

Characteristics:

- Maximum reliability
- Triple execution
- Majority voting enabled

This mode is activated using an **external control signal** or a **software-controlled register**, allowing the surrounding system to choose the appropriate operating mode.

---

## 12. Clock Gating and Operand Isolation

Instead of physically powering down unused hardware, redundant Execute units are disabled using standard digital design techniques.

### Clock Gating

Stops the clock signal from reaching inactive logic.



Benefits:

- Prevents unnecessary switching activity.
  - Reduces dynamic power consumption.
- 

## **Operand Isolation**

Blocks changing input signals from propagating into inactive Execute units.

Benefits:

- Prevents unnecessary internal transitions.
- Further reduces switching power.

Together, these techniques provide effective power savings while remaining compatible with standard ASIC synthesis flows.

---

## **13. Hazard Handling**

Since the processor uses pipelining, multiple instructions execute simultaneously.

This creates situations where instructions may interfere with one another.

The processor therefore includes:

### **Forwarding Unit**

Avoids unnecessary stalls by forwarding recently computed results directly to dependent instructions.

### **Stall Logic**

Temporarily pauses the pipeline when data dependencies cannot be resolved immediately.

### **Flush Logic**

Removes incorrectly fetched instructions after branches or jumps.

These mechanisms ensure correct program execution while maintaining pipeline efficiency.

---

## **14. Fault Model**

A fault model clearly defines the types of hardware faults that the processor is designed to handle.

## **Protected Faults**

### **Single Event Upsets (SEUs)**

- Register file bit flips
- Corrected using SEC-DED

### **Transient faults in Execute stage**

- Incorrect ALU outputs
  - Masked using TMR and majority voting
- 

## **Detected but Not Corrected**

### **Multiple Bit Upsets (MBUs)**

- Detected by SEC-DED
- Reported as an error
- Outside the correction capability of this design

Clearly defining the supported fault model establishes realistic design boundaries and simplifies verification.

---

# **15. Verification Strategy**

The processor will undergo comprehensive functional verification.

Testing includes:

- RTL simulation
- Pipeline verification
- Hazard testing
- SEC-DED verification
- TMR verification
- Majority voter verification
- Mode switching verification
- Fault injection campaigns

Fault injection involves deliberately introducing random bit flips into protected hardware to evaluate whether the implemented fault-tolerance mechanisms behave as expected.

Performance metrics such as fault coverage and correction effectiveness will be recorded and analyzed.

---

## 16. ASIC Implementation Flow

Following RTL verification, the processor will be implemented using a standard ASIC design flow.

The implementation stages include:

1. RTL Design
2. Functional Simulation
3. Logic Synthesis (Cadence Genus)
4. Gate-Level Verification
5. Floorplanning
6. Placement
7. Clock Tree Synthesis
8. Routing
9. Timing Analysis
10. Power Analysis
11. Physical Verification
12. Final Documentation

The design is intentionally **PDK-agnostic**, meaning it targets whichever standard-cell library is available in the laboratory rather than relying on a specific commercial fabrication process. This makes the project portable across different ASIC development environments.

---

## 17. Project Scope

### Included

- Custom RV32E processor
- Five-stage pipeline
- SEC-DED protected register file
- Triple Modular Redundancy on Execute stage
- Self-checking majority voter
- Adaptive redundancy controller
- Clock gating
- Operand isolation
- Fault injection verification

- ASIC synthesis and physical design flow
- 

## Excluded

To maintain a realistic and achievable scope, the following features are intentionally excluded:

- Full-core TMR
  - Custom ISA extensions
  - Branch prediction
  - Out-of-order execution
  - Multi-core processing
  - Formal safety certification (e.g., ISO 26262 or DO-254)
  - Advanced Design-for-Test (DFT) structures
  - Analog or mixed-signal circuitry
  - Fabrication and MPW submission unless supported by available funding and PDK access
- 

## 18. Expected Contributions

The project contributes a practical and educational architecture demonstrating how selective hardware redundancy can improve processor reliability without the excessive overhead associated with protecting an entire CPU.

Its key innovations include:

- A custom-built RV32E pipelined processor.
- Selective protection of only the most fault-sensitive hardware blocks.
- Integration of SEC-DED error correction with Execute-stage TMR.
- A self-checking majority voting mechanism.
- Adaptive redundancy control for balancing reliability and power consumption.
- A complete ASIC-oriented implementation and verification flow suitable for academic research.

This design demonstrates that reliability, power efficiency, and implementation complexity can be balanced through carefully targeted fault-tolerance mechanisms rather than indiscriminate hardware replication.

